

Dokumentasi dan Laporan Proyek Analisis Sentimen

Pendahuluan

Dokumen ini menyediakan panduan lengkap untuk aplikasi Analisis Sentimen dan Preprocessing NLP. Aplikasi ini adalah alat berbasis web yang memungkinkan pengguna untuk melakukan analisis sentimen pada teks, mulai dari pemrosesan awal (preprocessing), pelatihan model machine learning, hingga prediksi sentimen pada data baru. Laporan ini mencakup arsitektur, cara penggunaan, dan detail teknis dari setiap komponen aplikasi.

Tujuan Aplikasi

- Menyediakan *tool* berbasis web yang komprehensif untuk analisis sentimen.
- Memfasilitasi preprocessing teks dengan berbagai teknik (case folding, cleaning, tokenisasi, stopword removal, stemming).
- Memungkinkan pelatihan dan evaluasi model klasifikasi sentimen (Naive Bayes, Decision Tree, SVM).
- Mendukung prediksi sentimen real-time menggunakan model terbaik yang telah dilatih.
- Memungkinkan manajemen dataset (upload, pemilihan kolom) dan persistensi model.

Teknologi yang Digunakan

Backend

- **Python 3.13.6:** Bahasa pemrograman utama.
- **Flask 3.1.0:** *Web framework* untuk membangun API.
- **Pandas 2.2.3:** Untuk manipulasi dan analisis data.
- **NumPy 2.2.1:** Untuk komputasi numerik.
- **NLTK 3.9.1:** Pustaka untuk *Natural Language Toolkit* (tokenisasi dan penghapusan *stopwords*).
- **Sastrawi 1.0.1:** Pustaka untuk *stemming* Bahasa Indonesia.
- **Emoji 2.14.0:** Untuk pemrosesan emoji.
- **Scikit-learn:** Pustaka utama untuk ekstraksi fitur, model *machine learning* (Naive Bayes, Decision Tree, SVM), dan evaluasi.
- **Joblib:** Untuk persistensi (menyimpan dan memuat) model *machine learning*.

Frontend

- **Vue 3.5.24:** *Progressive JavaScript Framework* untuk membangun antarmuka pengguna.
- **Vite 7.2.2:** *Build tool* dan *dev server* yang cepat.
- **Bun:** *JavaScript Runtime* dan *Package Manager*.

Analisis & Visualisasi

- **Jupyter Notebook:** Untuk eksplorasi data, pengembangan pipeline NLP, dan analisis model secara transparan.
- **Matplotlib & Seaborn:** Untuk visualisasi data, seperti *confusion matrix* dan perbandingan performa model.

Sumber Data

Aplikasi ini memanfaatkan beberapa sumber data utama yang terletak di direktori `datasets/` :

- **GojekAppReviewV4.0.0-V4.9.3_Cleaned.csv** :
 - Merupakan dataset ulasan aplikasi Gojek yang telah dibersihkan.
 - Digunakan sebagai sumber data utama untuk melatih dan mengevaluasi model analisis sentimen. Dataset ini umumnya berisi kolom teks ulasan dan label sentimen (positif, negatif, netral).
- **linkDataSet.txt** :
 - File ini kemungkinan berisi tautan atau referensi ke dataset eksternal yang mungkin digunakan untuk tujuan yang lebih luas atau sebagai cadangan.

Dataset ini penting untuk pengembangan dan pengujian pipeline NLP serta model *machine learning* yang digunakan dalam aplikasi.

Cara Menjalankan Aplikasi

Aplikasi ini terdiri dari dua bagian utama: **Backend (Flask)** dan **Frontend (Vue.js)**. Untuk menjalankannya, ikuti langkah-langkah berikut:

Prasyarat

- **Python 3.10+**
- **Node.js** dan **Bun** (atau `npm` / `yarn`)

1. Setup Backend

Buka terminal dan navigasikan ke direktori `backend` .

```
cd backend

# Buat dan aktifkan virtual environment
python -m venv venv
source venv/bin/activate # Untuk Linux/macOS
# venv\Scripts\activate   # Untuk Windows

# Install semua dependensi Python
pip install -r requirements.txt
```

2. Setup Frontend

Buka terminal baru dan navigasikan ke direktori `frontend`.

```
cd frontend

# Install semua dependensi JavaScript menggunakan Bun
bun install
```

3. Menjalankan Server

Kami telah menyediakan skrip untuk menjalankan kedua server secara bersamaan.

- Untuk pengguna Linux/macOS:

```
chmod +x run.sh
./run.sh
```

- Untuk pengguna Windows:

```
run.bat
```

Setelah skrip dijalankan, aplikasi akan dapat diakses di `http://localhost:3000`.

Setelah skrip dijalankan, aplikasi akan dapat diakses di `http://localhost:3000`.

Penjelasan Lengkap Pipeline NLP

Berikut adalah penjelasan lengkap dan bertahap untuk keseluruhan pipeline *Natural Language Processing* (NLP) yang diimplementasikan dalam aplikasi ini, mulai dari data mentah hingga analisis hasil.

1. Preprocessing

Preprocessing adalah fondasi dari setiap pipeline NLP. Tujuannya adalah untuk membersihkan dan menstandarisasi teks agar model *machine learning* dapat memprosesnya secara efektif.

 Tampilan UI Preprocessing

Alur Preprocessing:

```
INPUT TEKS
↓
TEXT CLEANING (Lowercase, Hapus Emoji, URL, Mention, Angka, Tanda Baca)
↓
TOKENISASI (Memecah teks menjadi kata)
↓
STOPWORD REMOVAL (Menghapus kata umum)
↓
STEMMING (Mengubah kata ke bentuk dasar - Opsional)
↓
OUTPUT (Tokens bersih)
```

Detail Tahapan:

1. **Input Teks:** Menerima teks dari dataset atau input pengguna.

- **Contoh:** "Drivernya ramah banget & cepat sampai tujuan! Mantap Gojek! 🍑 #gojek"

2. **Text Cleaning:** Membersihkan teks dari elemen-elemen yang tidak relevan.

- **Hasil:** "drivernya ramah banget cepat sampai tujuan mantap gojek"

3. **Tokenisasi:** Memecah kalimat menjadi daftar kata (tokens).

- **Hasil:** ['drivernya', 'ramah', 'banget', 'cepat', 'sampai', 'tujuan', 'mantap', 'gojek']

4. **Stopword Removal:** Menghapus kata-kata umum yang tidak memiliki makna sentimen (misal: 'dan', 'di', 'ini').

- **Hasil:** ['drivernya', 'ramah', 'banget', 'cepat', 'sampai', 'tujuan', 'mantap', 'gojek'] (tidak ada stopwords pada contoh ini).

5. **Stemming (Optional):** Mengubah setiap kata ke bentuk dasarnya menggunakan library Sastrawi.

- **Hasil:** ['driver', 'ramah', 'banget', 'cepat', 'sampai', 'tuju', 'mantap', 'gojek']

2. Feature Extraction (Ekstraksi Fitur)

Setelah preprocessing, teks yang sudah menjadi daftar *tokens* bersih perlu diubah menjadi format numerik (vektor) yang dapat dipahami oleh model *machine learning*.

1. Bag-of-Words (BoW)

- **Konsep:** Merepresentasikan teks dengan menghitung frekuensi kemunculan setiap kata. Urutan kata diabaikan.
- **Implementasi:** Menggunakan `CountVectorizer` dari Scikit-learn.
- **Contoh:** Kalimat "driver cepat, driver ramah" akan menjadi vektor yang menandakan 'driver' muncul 2 kali, 'cepat' 1 kali, dan 'ramah' 1 kali.

2. TF-IDF (Term Frequency-Inverse Document Frequency)

- **Konsep:** Serupa dengan BoW, tetapi memberikan bobot yang lebih tinggi pada kata-kata yang sering muncul dalam satu dokumen tetapi jarang muncul di dokumen lain. Ini membantu mengidentifikasi kata-kata yang lebih penting dan informatif.
- **Implementasi:** Menggunakan `TfidfVectorizer` dari Scikit-learn.

3. Training

Pada tahap ini, data yang sudah berbentuk vektor digunakan untuk melatih model klasifikasi. Aplikasi ini melatih 3 jenis model dengan 2 metode ekstraksi fitur, sehingga total ada 6 kombinasi.

- **Data Split:** Dataset dibagi menjadi data latih (80%) dan data uji (20%) untuk memastikan evaluasi yang objektif.
- **Model yang Digunakan:**
 1. **Naive Bayes:** Model probabilistik yang cepat dan efektif untuk klasifikasi teks.
 2. **Decision Tree:** Model berbasis aturan (if-then-else) yang mudah diinterpretasi.
 3. **SVM (Support Vector Machine):** Model yang bertujuan menemukan “batas” terbaik untuk memisahkan antar kelas sentimen.

4. Evaluasi

Setelah dilatih, performa setiap model diukur menggunakan data uji.

- **Metrik Evaluasi:**
 - **Akurasi:** Persentase prediksi yang benar secara keseluruhan. $(TP + TN) / Total$.
 - **Presisi:** Dari semua yang diprediksi “Positif”, berapa persen yang benar-benar “Positif”. $TP / (TP + FP)$.
 - **Recall:** Dari semua yang seharusnya “Positif”, berapa persen yang berhasil diprediksi dengan benar. $TP / (TP + FN)$.
 - **F1-Score:** Rata-rata harmonik dari Presisi dan Recall, memberikan skor yang seimbang.
- **Confusion Matrix:** Sebuah tabel yang merangkum performa model dengan menunjukkan jumlah prediksi yang benar dan salah untuk setiap kelas. Di aplikasi, Anda akan melihat Confusion Matrix terpisah untuk setiap model, yang dihasilkan baik dari fitur **Bag-of-Words (BoW)** maupun **TF-IDF**. Ini membantu menganalisis jenis kesalahan prediksi yang dibuat oleh model untuk masing-masing metode ekstraksi fitur.
 - **True Positive (TP):** Prediksi “Positif”, Sebenarnya “Positif”.
 - **True Negative (TN):** Prediksi “Negatif”, Sebenarnya “Negatif”.
 - **False Positive (FP):** Prediksi “Positif”, Sebenarnya “Negatif”.
 - **False Negative (FN):** Prediksi “Negatif”, Sebenarnya “Positif”.

5. Prediksi

Setelah model terbaik dipilih berdasarkan hasil evaluasi (biasanya yang memiliki akurasi atau F1-Score tertinggi), model tersebut disimpan dan siap digunakan untuk prediksi.

- **Alur Prediksi:**

1. Teks baru dari pengguna diterima.
2. Teks tersebut melewati **pipeline preprocessing dan feature extraction yang sama persis** seperti saat pelatihan.
3. Vektor fitur yang dihasilkan diumpankan ke model terbaik yang telah disimpan.
4. Model mengeluarkan hasil prediksi sentimen beserta skor probabilitasnya.

6. Analisis Hasil dan Kesimpulan

Tahap terakhir adalah interpretasi hasil evaluasi untuk menarik kesimpulan.

- **Cara Menganalisis:**

- Lihat **tabel perbandingan hasil** di aplikasi. Model dengan metrik tertinggi (terutama Akurasi dan F1-Score) adalah kandidat model terbaik.
- Periksa **Confusion Matrix**. Jika model sering salah memprediksi kelas tertentu (misalnya, banyak kasus False Negative untuk sentimen "Netral"), ini menandakan model kesulitan mengenali kelas tersebut.

- **Kesimpulan:** Berdasarkan analisis, kita dapat menyimpulkan model mana yang paling cocok untuk dataset ini. Aplikasi ini secara otomatis memilih model dengan akurasi tertinggi sebagai **"model terbaik"** dan menyimpannya untuk fitur prediksi.

Arsitektur Aplikasi

Aplikasi ini menggunakan arsitektur **client-server**:

- **Backend (Server):**

- Dibangun dengan **Flask (Python)**.
- Menyediakan REST API untuk semua operasi NLP.
- Menggunakan **scikit-learn** untuk membangun dan melatih model.
- Menggunakan **Pandas** untuk manipulasi data.
- Menyimpan model terbaik yang telah dilatih menggunakan **Joblib** di direktori `backend/models/`.

- **Frontend (Client):**

- Dibangun dengan **Vue.js 3 (Composition API)** dan **Vite**.
- Menyediakan antarmuka pengguna (UI) yang interaktif.
- Berkomunikasi dengan backend melalui permintaan HTTP (`fetch`).

Fitur Aplikasi

1. Preprocessing Teks:

- Memproses dataset CSV atau teks tunggal.
- Menampilkan hasil setiap langkah preprocessing secara transparan.

2. Training & Evaluasi Model:

- Mengunggah dataset (format `.csv`).
- Memilih kolom teks dan label secara dinamis.
- Melatih tiga model klasifikasi: **Naive Bayes**, **Decision Tree**, dan **SVM**.
- Mengevaluasi model menggunakan metrik: Akurasi, Presisi, Recall, dan F1-Score.
- Menampilkan **Confusion Matrix** untuk analisis kesalahan.
- Secara otomatis menyimpan model dengan performa terbaik.

3. Prediksi Sentimen:

- Menggunakan model terbaik yang tersimpan untuk memprediksi sentimen dari teks baru.
- Menampilkan label prediksi (misalnya, "positif", "negatif") beserta probabilitasnya.

Contoh Penggunaan

Berikut adalah panduan lengkap penggunaan aplikasi, mulai dari melatih model hingga melakukan prediksi.

1. Melatih dan Mengevaluasi Model

Tujuan dari langkah ini adalah untuk melatih model-model *machine learning* pada dataset yang tersedia dan menyimpannya untuk digunakan dalam prediksi.

1. **Buka Aplikasi:** Pastikan aplikasi sudah berjalan dan akses `http://localhost:3000` di browser.

2. Pilih Tab “Training & Evaluasi”:

- Klik pada tab “Training & Evaluasi” untuk membuka halaman pelatihan model.

3. Konfigurasi Pelatihan:

- **Pilih Dataset:** Dari menu *dropdown*, pilih dataset yang akan digunakan. Contoh: `GojekAppReviewV4.0.0-V4.9.3_Cleaned.csv`.
- **Pilih Kolom Teks (Fitur):** Pilih kolom yang berisi teks ulasan, misalnya `review`.
- **Pilih Kolom Label (Target):** Pilih kolom yang berisi label sentimen, misalnya `sentiment`.
- **Aktifkan Stemming (Opsional):** Centang kotak “Gunakan Stemming” jika Anda ingin menerapkan proses stemming pada teks. Ini dapat meningkatkan akurasi tetapi akan membuat proses pelatihan lebih lama.



4. Mulai Pelatihan:

- Klik tombol “Mulai Training & Evaluasi”.
- Aplikasi akan memulai proses *backend* yang mencakup:
 - *Preprocessing* data.
 - Ekstraksi fitur (BoW dan TF-IDF).
 - Pelatihan tiga model: Naive Bayes, Decision Tree, dan SVM.
 - Evaluasi performa masing-masing model.

5. Analisis Hasil:

- Setelah selesai, halaman akan menampilkan:
 - **Tabel Perbandingan Hasil:** Menunjukkan metrik (Akurasi, Presisi, Recall, F1-Score) untuk setiap kombinasi model dan fitur.
 - **Model Terbaik:** Ringkasan model dengan akurasi tertinggi akan ditampilkan. Model ini secara otomatis disimpan di server (`backend/models/best_model.joblib`) untuk digunakan pada tahap prediksi.
 - **Confusion Matrix:** Visualisasi matriks untuk setiap model, membantu menganalisis kesalahan prediksi.



2. Melakukan Prediksi Sentimen

Setelah model terbaik disimpan, Anda dapat langsung menggunakannya untuk memprediksi sentimen teks baru.

1. Pilih Tab “Prediksi Teks”:

- Klik pada tab “**Prediksi Teks**” untuk pindah ke halaman prediksi.

2. Masukkan Teks:

- Di area input “**Masukkan teks untuk diprediksi**”, ketik atau tempel kalimat yang ingin Anda analisis.
 - **Contoh 1 (Positif):** "Drivernya cepat dan ramah, makanannya juga sampai dengan aman. Pertahankan!"
 - **Contoh 2 (Negatif):** "Sudah nunggu lama, eh drivernya cancel tiba-tiba. Kecewa banget."

3. Aktifkan Stemming (Opsional):

- Pastikan untuk mencentang atau tidak mencentang “**Gunakan Stemming**” sesuai dengan pengaturan saat model dilatih untuk hasil yang konsisten.

4. Mulai Prediksi:

- Klik tombol “**Prediksi Sentimen**”.

5. Lihat Hasil Prediksi:

- Hasil akan muncul di bawah tombol, menampilkan:
 - **Prediksi:** Sentimen yang diprediksi (misalnya, **positif** atau **negatif**).
 - **Probabilitas:** Skor kepercayaan untuk setiap kelas sentimen, membantu memahami seberapa yakin model dengan prediksinya.

 Hasil Prediksi Sentimen

Troubleshooting

- **“Backend tidak dapat terhubung”:** Pastikan server Flask (backend) sudah berjalan di `localhost:5000` tanpa error. Periksa terminal tempat Anda menjalankan `run.sh` atau `run.bat`.
- **Error saat upload file:** Pastikan file berformat `.csv` dan ukurannya tidak melebihi 16MB.
- **Model not trained:** Jika prediksi gagal, pastikan Anda telah melatih model terlebih dahulu melalui tab “Training & Evaluasi”.
- **Training Terasa Lambat:** Waktu training memang bisa terasa lama, terutama jika Anda mengaktifkan opsi “Gunakan Stemming”. Hal ini wajar dan disebabkan oleh beberapa faktor:
 - **Ukuran Dataset:** Semakin besar dataset, semakin banyak data yang harus diproses.
 - **Proses Stemming:** Stemming (mengubah kata ke bentuk dasar) adalah proses yang paling memakan waktu dalam pipeline preprocessing. Setiap kata dalam ribuan baris data harus dicocokkan dengan kamus Sastrawi.
 - **Jumlah Model:** Aplikasi ini melatih 6 kombinasi model (3 algoritma x 2 metode ekstraksi fitur).
 - **Solusi untuk Testing:** Untuk proses yang lebih cepat saat mencoba-coba, **coba jalankan training tanpa mencentang opsi “Gunakan Stemming”**. Anda akan melihat perbedaan kecepatan yang signifikan.

Penutup

Aplikasi ini menyediakan solusi end-to-end untuk analisis sentimen, cocok untuk tujuan edukasi maupun prototipe proyek. Dengan arsitektur yang modular, aplikasi ini dapat dikembangkan lebih lanjut, misalnya dengan menambahkan model-model baru atau teknik preprocessing yang lebih canggih.